



Exercise 9:

1. Demonstrate ZeroR technique on Iris dataset (by using necessary preprocessing technique(s)) and share your observations
2. Write a program to calculate chi-square value using Python. Report your observation.
3. Write a program of Naive Bayesian classification using Python programming language.

1. Demonstrate ZeroR technique on Iris dataset (by using necessary preprocessing technique(s)) and share your observations.

ZeroR (Zero Rule) Classifier Description:

ZeroR is the **simplest classification algorithm** — it's a basic benchmark to compare with more advanced models.

- **ZeroR (Zero Rule)** ignores all input features.
- It simply **predicts the majority class** (the most frequent label) in the training data.
- It doesn't learn any patterns — it just memorizes the most common outcome.

Example: If your dataset has this class distribution:

Class Label	Count
Yes	70
No	30

ZeroR will always predict "**Yes**" — because it's the majority class.

Advantages:

- Very simple
- Fast
- Serves as a **baseline** to evaluate if your real model is better than random guessing

Disadvantages:

- Does **not use any features** — so it's not a "real" classifier
- Cannot capture any relationships or patterns in data



When to use ZeroR:

- As a **baseline model**: If your actual classifier performs worse than ZeroR, something's wrong.
- For comparison to check if your model is really learning anything useful.

We are using Iris Data as a input file to ZeroR Classifier.

We use WEKA TOOL for performing ZeroR Classification.

Iris Data:

The **Iris dataset** is one of the most well-known and widely used datasets in statistics, machine learning, and pattern recognition. It was introduced by **Ronald A. Fisher** in 1936 as a classic example for **linear discriminant analysis**.

This dataset is used to classify flowers of the **Iris plant** into one of **three species** based on the measurements of their petals and sepals.

Note: We import or load the data set directly from Weka Tool/Software

Dataset Summary:

- **Total samples**: 150
- **Target classes**: 3 species of Iris flower
 - **Iris-setosa**
 - **Iris-versicolor**
 - **Iris-virginica**
- **Features**: 4 numeric attributes
- **No missing values**

Highlighting Features of Iris Dataset:

Feature Name	Description	Data Type	Units
Sepal Length	Length of the sepal	Float	Centimeters
Sepal Width	Width of the sepal	Float	Centimeters
Petal Length	Length of the petal	Float	Centimeters
Petal Width	Width of the petal	Float	Centimeters

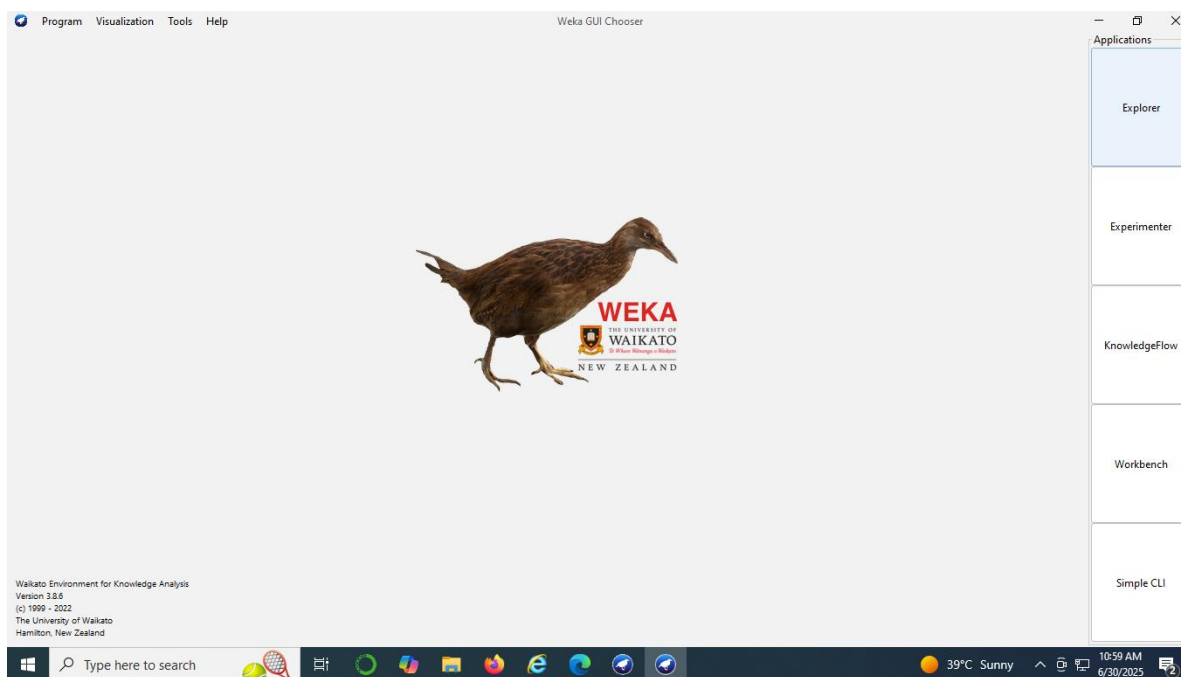
Each of the 150 samples includes all four features and a corresponding species label.

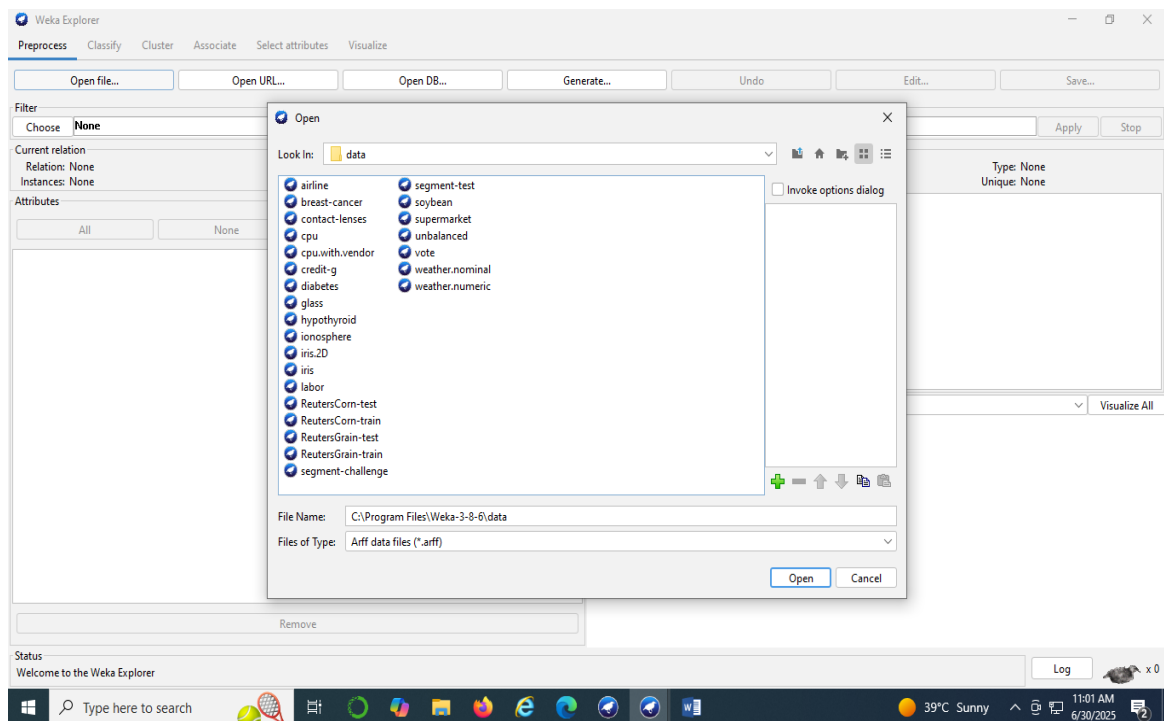
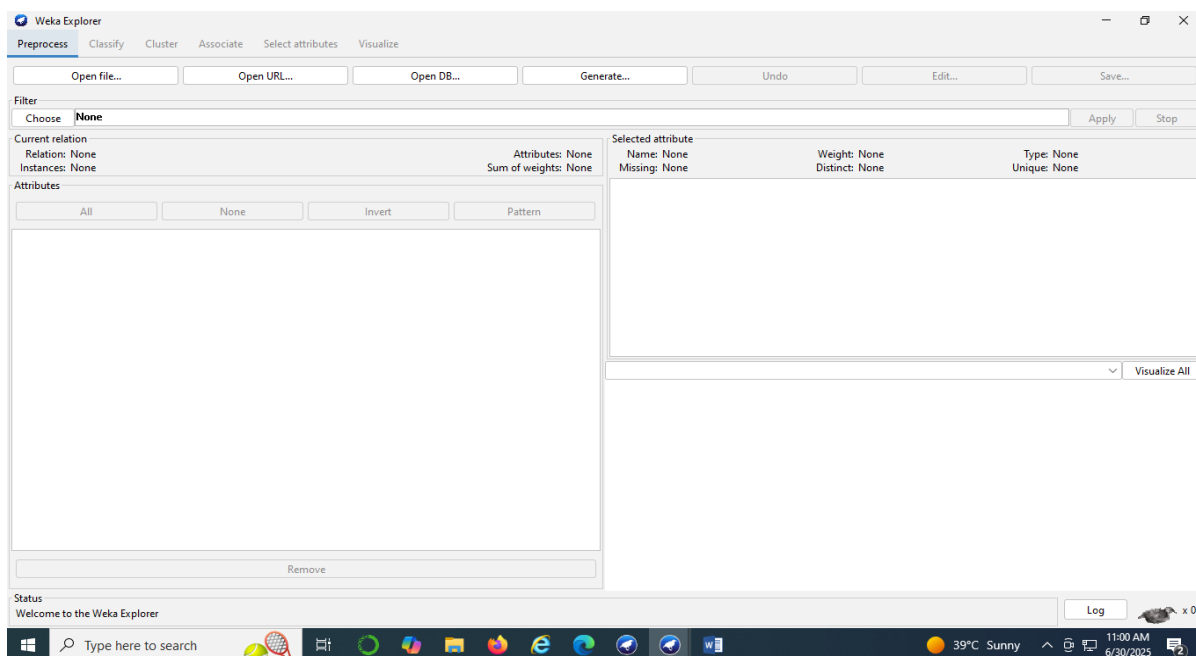


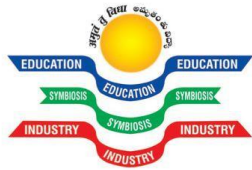
Key Characteristics:

- **Balanced dataset:** 50 samples for each species.
- **Perfect for classification** tasks.
- Used to test algorithms like:
 - Decision Tree
 - Naive Bayes
 - k-NN
 - Logistic Regression
- Ideal for **data visualization** and **pattern discovery**.
- Good for **educational purposes**, model evaluation, and feature selection.

Working of ZeroR:

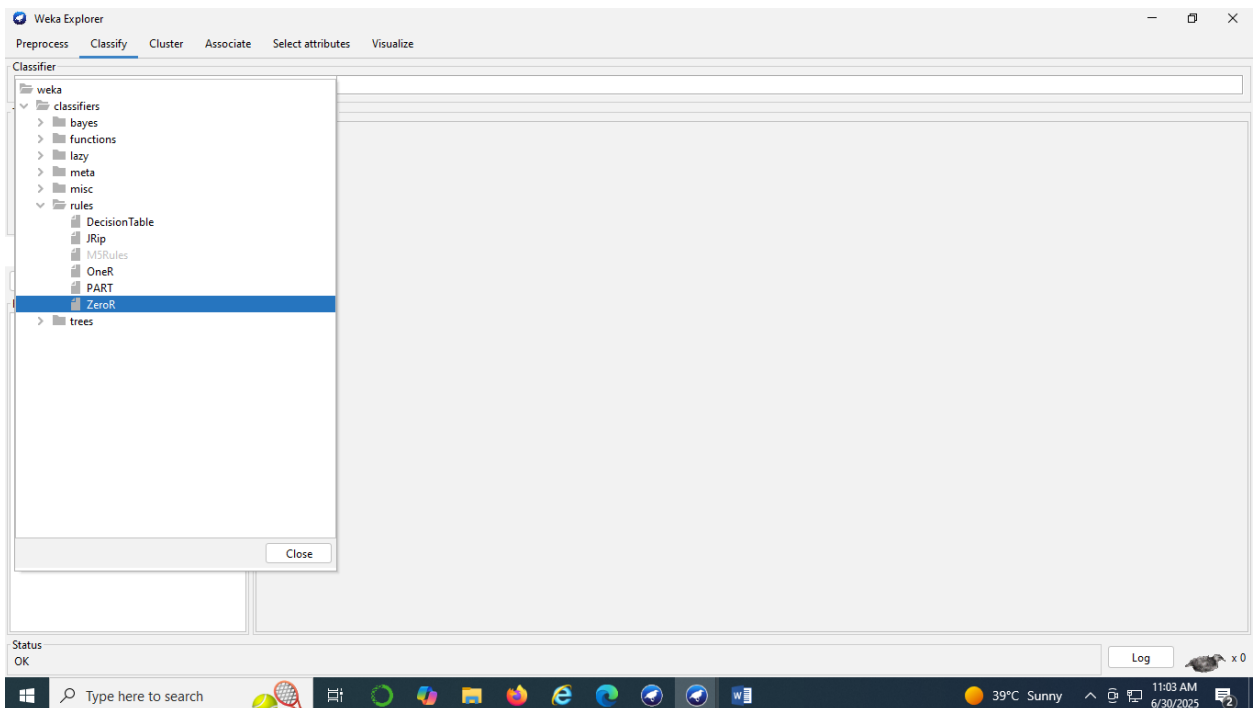
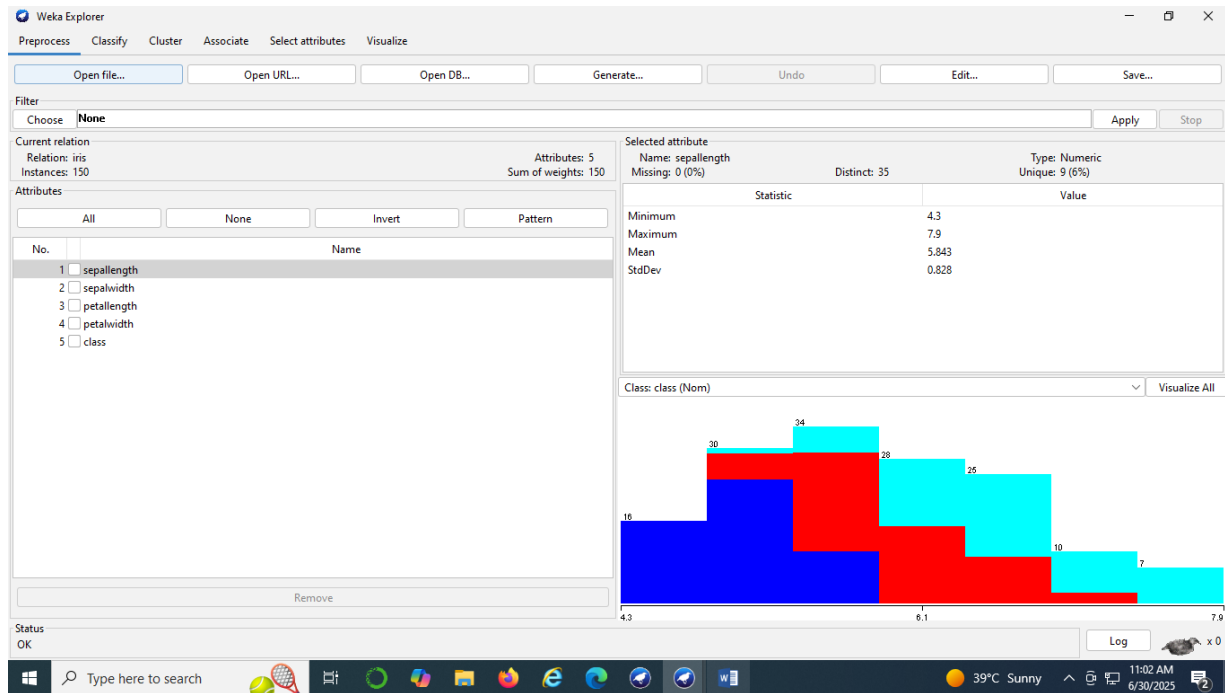
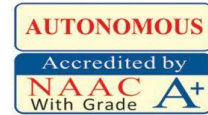






MIC College of Technology

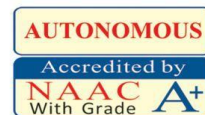
DVR & Dr. HS
ISO 9001:2015 Certified Institute
(Approved by AICTE & Permanently Affiliated to JNTUK, Kakinada)
Kanchikacherla - 521180, NTR Dist., A.P., India
Phones : 08678 - 273535 / 94914 57799 / 73826 16824
E mail: office@mictech.ac.in, Website: www.mictech.edu.in





MIC College of Technology

DVR & Dr. HS
ISO 9001:2015 Certified Institute
(Approved by AICTE & Permanently Affiliated to JNTUK, Kakinada)
Kanchikacherla - 521180, NTR Dist., A.P., India
Phones : 08678 - 273535 / 94914 57799 / 73826 16824
E mail: office@mictech.ac.in, Website: www.mictech.edu.in



Weka Explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Classifier: Choose ZeroR

Test options

- ☐ Use training set
- ☐ Supplied test set
- ☒ Cross-validation Folds: 10
- ☐ Percentage split %: 66

More options...

(Nom) class

Start Stop

Result list (right-click for options)

11:03:30 - rules.ZeroR

Classifier output

=== Run information ===

Scheme: weka.classifiers.rules.ZeroR
Relation: iris
Instances: 150
Attributes: 5
sepalength
sepalwidth
petalength
petalwidth
class

Test mode: 10-fold cross-validation

=== Classifier model (full training set) ===

ZeroR predicts class value: Iris-setosa

Time taken to build model: 0 seconds

=== Stratified cross-validation ===

=== Summary ===

Correctly Classified Instances	50	33.3333 %
Incorrectly Classified Instances	100	66.6667 %
Kappa statistic	0	
Mean absolute error	0.4444	
Root mean squared error	0.4714	
Relative absolute error	100 %	
Root relative squared error	100 %	
Total Number of Instances	150	

=== Detailed Accuracy By Class ===

Status: OK

Log

39°C Sunny 11:03 AM 6/30/2025



2. Write a program to calculate chi-square value using Python. Report your observation.

Chi-Square Test: The test indicates that there is a relationship between two entities. Handling data often involves testing hypotheses to extract useful information. In categorical analysis, chi-square tests are used to determine if observed patterns are likely to be purely random.

The **Chi-square value** is a statistical measure used to compare observed results with expected results. It is commonly used in hypothesis testing to assess whether distributions of categorical variables differ from each other.

In Python, you can compute the Chi-square value using the `scipy.stats` module.

Chi-square Test in Python(example program 1)

```
from scipy.stats import chi2_contingency

# Example: Observed data (contingency table)
observed = [[30, 10],
            [20, 40]]

# Perform Chi-square test
chi2, p, dof, expected = chi2_contingency(observed)

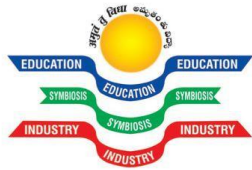
print("Chi-square value:", chi2)
print("Degrees of freedom:", dof)
print("Expected frequencies:\n", expected)
print("P-value:", p)
```

Output:

```
Chi-square value: 15.041666666666666
Degrees of freedom: 1
Expected frequencies:
[[20. 20.]
 [30. 30.]]
P-value: 0.00010516355403363098
```

Description :

- **Chi-square value (chi2):** Measures how much the observed data deviate from the expected.
- **Degrees of freedom (dof):** Based on the table size.
- **P-value:** Used to decide if the difference is statistically significant.
 - If $p < 0.05$, you typically reject the null hypothesis.



For example, (testing gender vs product preference)?

In this **example** using the Chi-square test in Python — we'll test whether **gender and product preference** are related.

Scenario:

A company surveyed 100 people on their **preferred product (A or B)** and recorded their **gender**.

	Product A	Product B
Male	30	20
Female	10	40

We want to test if **product preference is independent of gender**.

Chi-square Test with Python(Example Program 2)

```
from scipy.stats import chi2_contingency

# Observed data: Gender vs Product preference
observed = [
    [30, 20], # Male
    [10, 40]  # Female
]

# Perform Chi-square test
chi2, p, dof, expected = chi2_contingency(observed)

# Output results
print("Chi-square value:", chi2)
print("Degrees of freedom:", dof)
print("Expected frequencies:\n", expected)
print("P-value:", p)

# Interpretation
if p < 0.05:
    print("Reject the null hypothesis: There is a significant relationship between gender and product preference.")
else:
    print("Fail to reject the null hypothesis: No significant relationship found.")
```

OUTPUT

Chi-square value: 15.041666666666668
Degrees of freedom: 1
Expected frequencies:



```
[[20. 30.]
```

```
[20. 30.]]
```

P-value: 0.00010516355403363114

Reject the null hypothesis: There is a significant relationship between gender and product preference.

Interpretation:

- **Null hypothesis (H_0):** Gender and product preference are independent.
- If the **p-value** is less than 0.05, we conclude that **gender likely influences product preference**.

if you want to visualize this with a heatmap or bar chart!

A **heatmap** is a **graphical representation of data** where individual values are shown as colors. It is commonly used to show the **relationship or correlation between variables**.

Example Program3:

```
import seaborn as sns

import matplotlib.pyplot as plt

from scipy.stats import chi2_contingency

import numpy as np

import pandas as pd

# Data

observed = np.array([[30, 20],

                     [10, 40]])

labels = ["Male", "Female"]

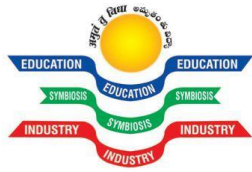
products = ["Product A", "Product B"]

# Chi-square test

chi2, p, dof, expected = chi2_contingency(observed)

# Convert to DataFrames

observed_df = pd.DataFrame(observed, index=labels, columns=products)
```



```
expected_df = pd.DataFrame(expected, index=labels, columns=products)

# Set style

sns.set_style("whitegrid")

plt.rcParams.update({'font.size': 12, 'axes.titlesize': 14})

# Plot 1: Heatmaps

plt.figure(figsize=(12, 5))

# Observed heatmap

plt.subplot(1, 2, 1)

sns.heatmap(observed_df, annot=True, fmt="d", cmap="YlGnBu", cbar=False)

plt.title("Observed Frequencies")

plt.xlabel("Product")

plt.ylabel("Gender")

# Expected heatmap

plt.subplot(1, 2, 2)

sns.heatmap(expected_df, annot=True, fmt=".1f", cmap="YlOrBr", cbar=False)

plt.title("Expected Frequencies")

plt.xlabel("Product")

plt.ylabel("Gender")

plt.tight_layout()

plt.savefig("heatmaps_gender_product.png", dpi=300) # Save heatmap image

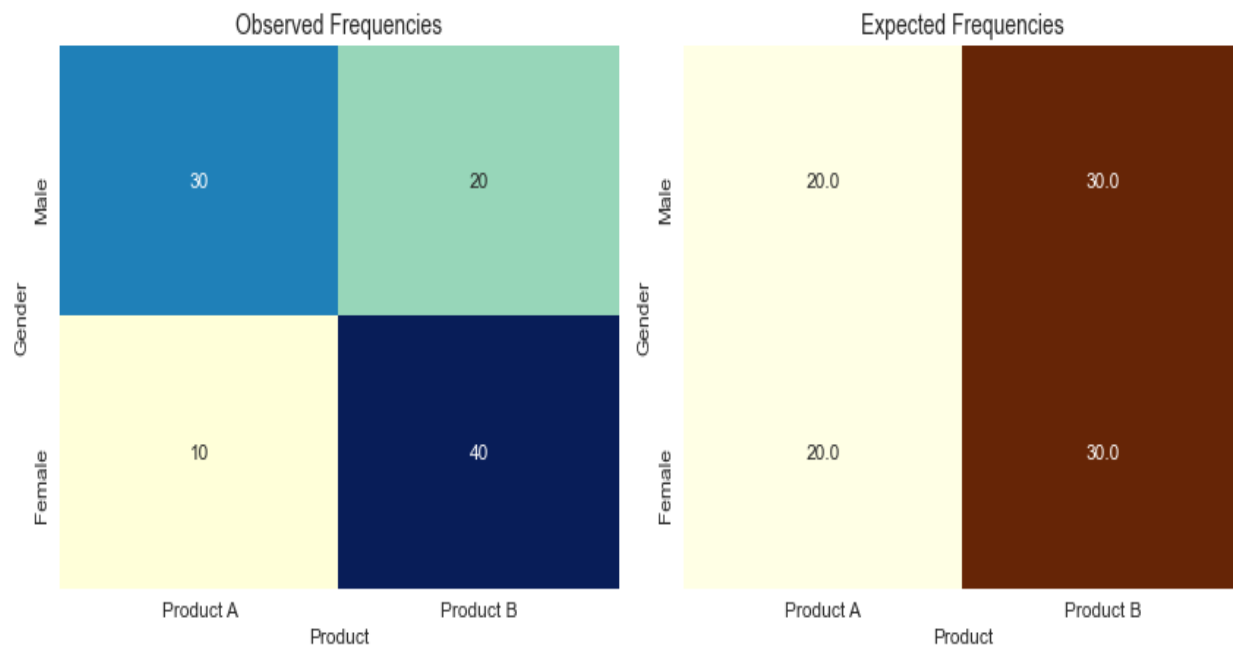
plt.show()

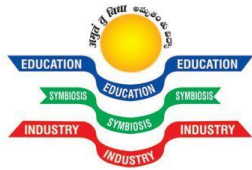
# Plot 2: Bar Chart

observed_df.plot(kind="bar", figsize=(8, 5), color=["#1f77b4", "#ff7f0e"])
```

```
plt.title("Gender vs Product Preference")
plt.xlabel("Gender")
plt.ylabel("Number of People")
plt.xticks(rotation=0)
plt.grid(axis='y')
plt.tight_layout()
plt.savefig("bar_chart_gender_product.png", dpi=300) # Save bar chart image
plt.show()
```

OUTPUT:







3. Write a program of Naive Bayesian classification using Python programming language.

Naive Bayesian Classifier:

Naive Bayes is a simple and powerful **probabilistic machine learning algorithm** based on **Bayes' Theorem**. It is called “naive” because it **assumes all features are independent** of each other, which is rarely true in real data—but it still works well in many practical cases.

Bayes' Theorem (The Foundation):

$$P(C|X)=P(X)P(X|C) \cdot P(C)$$

Where:

- $P(C|X)P(C|X)P(C|X)$: Probability of class **C** given the features **X**
- $P(X|C)P(X|C)P(X|C)$: Probability of features **X** given class **C**
- $P(C)P(C)P(C)$: Prior probability of class **C**
- $P(X)P(X)P(X)$: Prior probability of features **X**

Key Concepts of Naïve Bayes:

- **Supervised learning**: You train it on labeled data.
- **Fast and efficient**, even with large datasets.
- Works best for:
 - **Text classification** (e.g., spam filtering, sentiment analysis)
 - **Medical diagnosis**
 - **Document categorization**

Types of Naive Bayes:

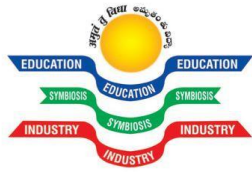
1. **Gaussian Naive Bayes**: For continuous data (assumes Gaussian distribution).
2. **Multinomial Naive Bayes**: For discrete count data (e.g., word counts in documents).
3. **Bernoulli Naive Bayes**: For binary/boolean features (e.g., word present or not).

Advantages:

- Simple to implement
- Requires less training data
- Performs well with high-dimensional data (e.g., text)
- Handles both binary and multiclass classification

Limitations:

- Assumes **independence** between features (which is often unrealistic)
- May perform poorly when feature interactions are important



Program:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Load Dataset

iris = load_iris()

X = iris.data    # Features (sepal length, petal length, etc.)

y = iris.target  # Labels (species)

# 2. Split into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 3. Create and train Naive Bayes classifier

model = GaussianNB()

model.fit(X_train, y_train)

# 4. Make predictions

y_pred = model.predict(X_test)

# 5. Evaluate the model

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```



OUTPUT:

Accuracy: 0.9777777777777777

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	0.92	0.96	13
2	0.93	1.00	0.96	13

accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

Confusion Matrix:

[[19 0 0]

[0 12 1]

[0 0 13]]

Explanation:

- **GaussianNB:** Used when features are continuous and normally distributed (as in Iris data).
- **Accuracy:** Percentage of correct predictions.
- **Confusion Matrix:** Shows correct vs incorrect classifications by class.
- **Classification Report:** Precision, recall, F1-score for each class.